

WEEK 2 TASK

DATA COLLECTION, CLEANING, AND PREPROCESSING FOR LOGISTICS ANALYSIS

Python • Pandas • Data Quality • Logistics Analytics • [Github\(link\)](#)

1. Objective

The objective of this task is to simulate a logistics data-preprocessing pipeline. The workflow covers data collection, data-quality assessment, missing-value treatment, duplicate removal, categorical standardization, outlier handling, feature engineering, and normalization so that the final dataset is suitable for analytics and further modeling.

2. Dataset Selection and Data Collection Simulation

The reference dataset is the publicly available DataCo Smart Supply Chain dataset on Kaggle. It contains supply-chain transaction information including shipping performance, sales, customer/product attributes, delivery status, and shipping mode. For this assignment, a smaller 1,010-row logistics dataset was simulated with similar operational fields so the complete preprocessing process can be demonstrated quickly and transparently.

Reference: DataCo SMART SUPPLY CHAIN FOR BIG DATA ANALYSIS (Kaggle).

Public dataset characteristics: approximately 180,519 records and 53 columns in the commonly used DataCo file.

3. Simulated Dataset Characteristics

Field	Type	Purpose
Shipment_ID	Categorical/ID	Unique shipment identifier
Order_Date	Date	Shipment/order time reference
Region	Categorical	Operational/geographical segment
Shipping_Mode	Categorical	Road, Rail, Air, or Ship
Vehicle_Type	Categorical	Vehicle/resource type
Distance_km	Numeric	Transport distance
Package_Weight_kg	Numeric	Shipment weight

Expected_Delivery_Days	Numeric	Planned delivery duration
Delivery_Days	Numeric	Actual delivery duration
Shipping_Cost_INR	Numeric	Transportation cost
Delivery_Rating	Numeric	Customer/service rating
Weather	Categorical	Operating weather condition
Delivery_Status	Categorical	Operational delivery status

4. Data Quality Issues Identified

- Missing values were intentionally introduced in important categorical and numeric columns.
- Duplicate shipment records were added to test duplicate detection.
- Categorical values contained inconsistent capitalization and extra spaces.
- Extreme values were inserted into distance, package weight, and shipping cost to simulate outliers.
- Numeric columns required type conversion and validation before analysis.

5. Cleaning and Preprocessing Methodology

Technique	Python Method	Reason
Duplicate removal	drop_duplicates()	Prevents the same shipment from being counted more than once.
Categorical standardization	str.strip().str.title()	Makes labels consistent and prevents false categories.
Numeric conversion	pd.to_numeric(..., errors='coerce')	Converts invalid numeric entries to missing values for controlled treatment.
Missing numeric values	Median imputation	Median is robust to skew and extreme values.
Missing categorical values	Mode imputation	Uses the most common operational category.
Outlier treatment	IQR capping	Limits extreme observations without deleting potentially useful shipments.
Date conversion	pd.to_datetime()	Enables time-based analysis and feature engineering.
Feature engineering	Delay and Late_Flag	Creates logistics KPIs directly useful for performance analysis.

Normalization

Min-Max scaling

Puts selected numeric variables on a comparable 0–1 scale.

6. Python Implementation

```
import pandas as pd
import numpy as np
```

1. Load data

```
df = pd.read_csv("logistics_raw_data.csv")
```

2. Inspect structure and missing values

```
print(df.shape)
print(df.info())
print(df.isnull().sum())
```

3. Remove duplicates

```
df = df.drop_duplicates()
```

4. Standardize categorical fields

```
cat_cols = ["Region", "Shipping_Mode", "Vehicle_Type",
            "Weather", "Delivery_Status"]

for col in cat_cols:
    df[col] = df[col].astype("string").str.strip().str.title()
```

5. Handle missing values

```
num_cols = ["Distance_km", "Package_Weight_kg",
            "Expected_Delivery_Days", "Delivery_Days",
            "Shipping_Cost_INR", "Delivery_Rating"]

for col in num_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")
    df[col] = df[col].fillna(df[col].median())

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
```

6. Detect and cap outliers using IQR

```
for col in ["Distance_km", "Package_Weight_kg",
            "Shipping_Cost_INR", "Delivery_Days"]:
    q1, q3 = df[col].quantile([0.25, 0.75])
```

```

iqr = q3 - q1

lower = q1 - 1.5 * iqr

upper = q3 + 1.5 * iqr

df[col] = df[col].clip(lower, upper)

```

7. Feature engineering

```

df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")

df["Delivery_Delay_Days"] = (

    df["Delivery_Days"] - df["Expected_Delivery_Days"]

).round(1)

df["Late_Flag"] = (

    df["Delivery_Days"] > df["Expected_Delivery_Days"]

).astype(int)

```

8. Min-Max normalization

```

for col in ["Distance_km", "Package_Weight_kg",

    "Shipping_Cost_INR", "Delivery_Days"]:

    df[col + "_Normalized"] = (

        (df[col] - df[col].min()) /

        (df[col].max() - df[col].min()))

```

9. Save final dataset

```

df.to_csv("logistics_cleaned_data.csv", index=False)

```

7. Validation After Cleaning

After preprocessing, the cleaned file contains unique shipment records, standardized categories, imputed missing values, controlled numeric outliers, converted date fields, and additional logistics indicators such as `Delivery_Delay_Days` and `Late_Flag`. A final validation should confirm that required columns contain no unexpected missing values and that numeric ranges are reasonable.

```

print(df.shape)

print("Missing values:", df.isnull().sum().sum())

print("Duplicate rows:", df.duplicated().sum())

print(df.describe())

```

8. Reflection and Business Impact

Data quality directly affects logistics decisions. Duplicate shipments can inflate order volume and cost, missing values can distort averages, inconsistent category labels can split one operational segment into several false groups, and extreme values can dominate statistical analysis. A structured preprocessing pipeline produces more reliable KPIs and supports better decisions about delivery performance, transportation cost, capacity planning, and service quality.

9. Final Deliverables

- `logistics_raw_data.csv` – raw simulated dataset containing realistic data-quality problems.
- `logistics_cleaned_data.csv` – cleaned and preprocessed dataset ready for analysis.
- `preprocessing_summary.csv` – short summary of preprocessing checks.
- Python code – documented pipeline included in this report.

10. Conclusion

The task demonstrates an end-to-end logistics data-preprocessing workflow using Python and Pandas. By systematically identifying and correcting data-quality problems, the resulting dataset becomes more consistent, comparable, and suitable for exploratory analysis, visualization, KPI calculation, and future predictive modeling.